

JOUR

03

sur 05

DATE DE SESSION

lundi 1er juin 2026

Durée : 4h en présentiel

Structures, fonctions & pointeurs

Programme de la journée

Structures de contrôle : if, switch, while, for

Fonctions : définition, paramètres, retour

Tableaux, chaînes de caractères, introduction aux pointeurs

Atelier — petit jeu "devine le nombre"

Aujourd'hui : la programmation pour de vrai

Faire des choix, répéter, organiser, manipuler la mémoire

Vous savez compiler un programme. Vous savez déclarer des variables. Maintenant on va apprendre à faire faire des choses intéressantes à votre programme : prendre des décisions, répéter des actions, organiser le code en blocs réutilisables.

01

Décider

if/else, switch — exécuter du code seulement si une condition est vraie.

02

Répéter

while, for, do-while — exécuter du code plusieurs fois.

03

Découper

Fonctions — encapsuler un comportement nommé et le réutiliser.

04

Stocker en série

Tableaux et chaînes — manipuler plusieurs valeurs ensemble.

05

Adresser

Pointeurs — manipuler directement les adresses mémoire.

Faire des choix : if / else if / else

La structure conditionnelle de base

main.c — gcc main.c -o main

```
#include <stdio.h>

int main(void) {
    int note;
    printf("Note (0-20) : ");
    scanf("%d", &note);

    if (note < 0 || note > 20) {
        printf("Note invalide\n");
    } else if (note >= 16) {
        printf("Tres bien\n");
    } else if (note >= 12) {
        printf("Bien\n");
    } else if (note >= 10) {
        printf("Passable\n");
    } else {
        printf("Insuffisant\n");
    }
    return 0;
}
```

- Les conditions sont évaluées dans l'ordre : dès qu'une est vraie, son bloc est exécuté et on saute la fin.
- Les accolades { } délimitent les blocs. Si le bloc est sur une seule ligne, elles sont optionnelles — mais il vaut mieux les mettre toujours pour éviter les pièges.
- Attention : == est la comparaison, = est l'affectation. Confondre les deux est une source classique de bugs.

Le switch / case

Quand on a beaucoup de cas exacts à gérer

```
main.c — gcc main.c -o main

#include <stdio.h>

int main(void) {
    int jour;
    printf("Numero du jour (1-7) : ");
    scanf("%d", &jour);

    switch (jour) {
        case 1: printf("Lundi\n");    break;
        case 2: printf("Mardi\n");   break;
        case 3: printf("Mercredi\n"); break;
        case 4: printf("Jeudi\n");    break;
        case 5: printf("Vendredi\n"); break;
        case 6:
        case 7:
            printf("Week-end\n");    break;
        default:
            printf("Jour invalide\n");
    }
    return 0;
}
```

- Le switch ne fonctionne qu'avec des valeurs entières (int, char, énumérations) — pas avec des chaînes ou des doubles.
- Le mot-clé break est essentiel : sans lui, l'exécution continue dans le case suivant (le "fall-through").
- Le default attrape toutes les valeurs non listées — équivalent du "sinon" final.
- Plusieurs case peuvent partager le même bloc (case 6: case 7: → même comportement).

Boucles while et do...while

Répéter tant qu'une condition est vraie

```
main.c - gcc main.c -o main

#include <stdio.h>

int main(void) {
    // while : test AVANT chaque execution
    int compteur = 1;
    while (compteur <= 5) {
        printf("Tour %d\n", compteur);
        compteur++;           // sans cela : boucle infinie !
    }

    // do...while : exécute au moins une fois
    int reponse;
    do {
        printf("Entrez 0 pour quitter : ");
        scanf("%d", &reponse);
    } while (reponse != 0);

    printf("Au revoir\n");
    return 0;
}
```

- while : vérifie la condition AVANT chaque exécution. Si fausse dès le début → 0 itération.
- do...while : exécute le bloc UNE fois minimum, puis vérifie la condition. Pratique pour les saisies utilisateur.
- Oublier d'incrémenter le compteur = boucle infinie. Ctrl+C dans le terminal pour arrêter.

La boucle for

La plus utilisée : tout est concentré sur une seule ligne

```
main.c — gcc main.c -o main

#include <stdio.h>

int main(void) {
    // Afficher les nombres de 1 à 10
    for (int i = 1; i <= 10; i++) {
        printf("%d ", i);
    }
    printf("\n");

    // Calculer la somme de 1 à 100
    int somme = 0;
    for (int i = 1; i <= 100; i++) {
        somme += i;           // somme = somme + i
    }
    printf("Somme 1+2+...+100 = %d\n", somme);

    // Compte à rebours
    for (int i = 10; i > 0; i--) {
        printf("%d... ", i);
    }
    printf("Decollage !\n");
    return 0;
}
```

- Le for prend 3 expressions séparées par des ; : initialisation, condition, incrément.
- `for (int i = 0; i < n; i++)` — la déclaration de `i` dans le for limite sa portée à la boucle (depuis C99).
- Le for est juste un raccourci syntaxique pour un while ; les deux peuvent toujours s'échanger.

Les fonctions — découper son code

Une fonction = un bloc de code nommé, paramétré, réutilisable

```
main.c — gcc main.c -o main

#include <stdio.h>

// Prototype : annonce la fonction au compilateur
int carre(int x);

// Definition : implementation réelle
int carre(int x) {
    return x * x;
}

int main(void) {
    int a = carre(5);           // a = 25
    int b = carre(7);           // b = 49
    printf("5*5=%d, 7*7=%d\n", a, b);
    printf("12*12=%d\n", carre(12));
    return 0;
}
```

- Anatomie : `type_retour nom(paramètres) { ... return valeur; }`
- Le prototype (déclaration sans corps) permet d'utiliser la fonction avant sa définition. Indispensable quand on a plusieurs fichiers.
- `void` en type de retour = la fonction ne retourne rien (équivalent de "procédure" en algorithmie).

Paramètres, valeur de retour, portée

Les règles de communication entre fonctions

Une fonction reçoit ses entrées via les paramètres Listés entre parenthèses. Ce sont des variables locales initialisées avec les arguments passés à l'appel.

Elle peut retourner UNE valeur via return L'exécution s'arrête au return. Pas de return = retourne implicitement (sauf int main qui rend 0).

void en paramètres = pas de paramètres int f(void) signifie « la fonction f n'attend rien ». Équivalent à int f() en pratique.

Variables locales Déclarées dans une fonction, elles n'existent que pendant l'exécution de la fonction. Détruites à la sortie.

Variables globales Déclarées HORS de toute fonction : visibles partout. À utiliser avec parcimonie — elles compliquent le débogage.

Une fonction = une responsabilité Bonne pratique : si vous n'arrivez pas à donner un nom court à votre fonction, c'est qu'elle en fait trop.

Passage par valeur — le piège classique

Pourquoi cette fonction ne fait rien ?

```
main.c — gcc main.c -o main

#include <stdio.h>

void doubler(int x) {
    x = x * 2;
    // x est une COPIE locale, modifier x ne change rien dehors
}

int main(void) {
    int valeur = 10;
    doubler(valeur);
    printf("valeur = %d\n", valeur); // affiche 10 (et non 20 !)
    return 0;
}
```

- En C, les paramètres sont passés PAR VALEUR : la fonction reçoit une copie, pas l'original.
- Modifier le paramètre dans la fonction n'affecte PAS la variable du programme appelant.
- Pour modifier une variable de l'extérieur, il faudra utiliser des pointeurs (qu'on découvre dans quelques slides).

Les tableaux — stocker plusieurs valeurs

Une suite de cases mémoire de même type

```
main.c — gcc main.c -o main

#include <stdio.h>

int main(void) {
    int notes[5] = { 14, 17, 9, 12, 18 };

    // Acces par indice (commence a 0 !)
    printf("Premiere note : %d\n", notes[0]); // 14
    printf("Derniere note : %d\n", notes[4]); // 18

    // Parcours classique avec for
    int total = 0;
    for (int i = 0; i < 5; i++) {
        total += notes[i];
    }
    printf("Moyenne : %.2f\n", total / 5.0);

    // Tableau a taille fixe declare puis rempli
    double prix[3];
    prix[0] = 9.99;
    prix[1] = 14.50;
    prix[2] = 7.25;
    return 0;
}
```

- Les indices commencent à 0. Un tableau de taille n a des indices de 0 à $n-1$.
- Accéder à `notes[5]` dans un tableau de taille 5 = comportement indéfini (vous lisez de la mémoire qui ne vous appartient pas).
- La taille d'un tableau classique en C est FIXE à la compilation — pas de redimensionnement dynamique sans `malloc`.

Chaînes de caractères en C

Un tableau de char terminé par '\0'

```
main.c - gcc main.c -o main

#include <stdio.h>
#include <string.h>           // pour strlen, strcpy, strcmp

int main(void) {
    char prenom[20] = "Ayoub"; // taille 20, contient 'A','y','o','u','b','\0'

    printf("Bonjour %s\n", prenom);
    printf("Longueur : %zu\n", strlen(prenom)); // 5

    char autre[20];
    strcpy(autre, "Adam"); // copier
    if (strcmp(prenom, autre) == 0) {
        printf("Memes prenoms\n");
    } else {
        printf("Différents\n");
    }

    // Lecture clavier
    char saisie[30];
    printf("Tapez quelque chose : ");
    scanf("%29s", saisie); // pas de & pour une chaîne
    printf("Vous avez tape : %s\n", saisie);
    return 0;
}
```

- Une chaîne en C = tableau de char + un caractère '\0' à la fin (le "null terminator").
- Le %s de printf affiche jusqu'au '\0'. Si vous oubliez ce '\0', printf affichera n'importe quoi.
- string.h fournit strlen (longueur), strcpy (copie), strcmp (comparaison), strcat (concaténation).

Les pointeurs — la singularité du C

Le concept qui fait peur, mais qui est en fait simple

Un pointeur est une variable qui stocke une adresse mémoire.

Toutes les variables sont stockées quelque part dans la mémoire vive. Chaque emplacement a une adresse (un grand nombre, par exemple 0x7ffec3a8b0c). Un pointeur est simplement une variable qui contient l'adresse d'une autre variable. Cela permet de manipuler indirectement les données : passer une référence à une fonction (au lieu d'une copie), parcourir un tableau, allouer dynamiquement (Jour 4), construire des structures complexes.

Exemples concrets

Variable normale

`int x = 42;` — x stocke une valeur (42).

Pointeur

`int *p = &x;` — p stocke l'ADRESSE de x.

Déréférencement

*p donne la VALEUR à l'adresse stockée dans p (donc 42).

Les pointeurs en code

Trois opérateurs à maîtriser : * & *

```
main.c - gcc main.c -o main

#include <stdio.h>

int main(void) {
    int x = 42;
    printf("Adresse de x : %p\n", (void*)&x);
    printf("Valeur de x : %d\n", x);

    // Declaration d'un pointeur
    int *p = &x;          // p contient l'adresse de x

    // * = dereferencement (accéder à la valeur pointée)
    printf("*p vaut : %d\n", *p); // 42

    // Modifier x via le pointeur
    *p = 100;
    printf("Maintenant x = %d\n", x); // 100 (modifié via *p)
    return 0;
}
```

- & devant une variable = donne son adresse mémoire.
- * à la déclaration = signale qu'on déclare un POINTEUR. *p à l'utilisation = donne la VALEUR pointée.
- Un pointeur a un type : int *p ne peut pointer que vers un int. Ce type aide le compilateur à savoir combien d'octets lire.

Passage par adresse — modifier de l'extérieur

Avec un pointeur, on peut enfin modifier les paramètres

```
main.c — gcc main.c -o main

#include <stdio.h>

// Recoit L'ADRESSE d'un int, peut donc Le modifier
void doubler(int *x) {
    *x = *x * 2;
}

// Echanger deux valeurs : impossible sans pointeurs
void echanger(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

int main(void) {
    int valeur = 10;
    doubler(&valeur);           // on passe l'adresse
    printf("valeur = %d\n", valeur); // 20

    int x = 1, y = 2;
    echanger(&x, &y);
    printf("x=%d, y=%d\n", x, y); // x=2, y=1
    return 0;
}
```

- Pour passer "par référence" en C, on passe en réalité l'ADRESSE de la variable.
- C'est exactement ce que fait scanf : on écrit `scanf("%d", &age)` pour qu'il puisse écrire dans `age`.
- C++ ajoutera la "vraie" référence (`int &x`) qui simplifie la syntaxe — on en parlera demain.

Pointeurs et tableaux — la même chose ?

Une équivalence à connaître pour bien comprendre le C

Le nom d'un tableau est en réalité un pointeur sur sa première case `int t[10];` — `t` est équivalent à `&t[0]`. Le compilateur "décale" automatiquement selon l'indice.

`t[i]` est strictement équivalent à `*(t + i)` Quand vous écrivez `t[3]`, le compilateur calcule l'adresse de `t + 3` cases puis déréférence.

Conséquence : un tableau passé à une fonction est passé par adresse `void f(int t[10])` — `t` est en fait un pointeur. La fonction peut modifier le tableau de l'appelant !

On perd l'information de taille Dans la fonction, `sizeof(t)` donne la taille d'un pointeur, pas du tableau. Il faut passer la taille en paramètre.

Exemple typique : `void afficher(int t[], int taille)` On boucle de 0 à `taille-1` en lisant `t[i]`.

À RETENIR

tableau $\approx$ pointeur sur la première case

$t[i] \Leftrightarrow *(t + i)$

En paramètre de fonction, écrire `int t[]` ou `int *t` est équivalent.

Synthèse du jour — moyenne de notes

On combine boucles, fonctions, tableaux et pointeurs

```
main.c — gcc main.c -o main

#include <stdio.h>

double moyenne(int t[], int n) {
    int somme = 0;
    for (int i = 0; i < n; i++) somme += t[i];
    return (double)somme / n;
}

void min_max(int t[], int n, int *pmin, int *pmax) {
    *pmin = *pmax = t[0];
    for (int i = 1; i < n; i++) {
        if (t[i] < *pmin) *pmin = t[i];
        if (t[i] > *pmax) *pmax = t[i];
    }
}

int main(void) {
    int notes[6] = { 14, 17, 9, 12, 18, 11 };
    int mn, mx;
    min_max(notes, 6, &mn, &mx);
    printf("Min=%d, Max=%d, Moy=%.2f\n", mn, mx, moyenne(notes, 6));
    return 0;
}
```

- min_max retourne 2 valeurs grâce à 2 paramètres pointeurs (en C il n'y a pas de tuples).
- Le cast (double) force la division à donner un résultat à virgule.
- Cette structure (parcours + accumulation) est partout en C — entraînez-vous à la reproduire.

Atelier 3 — Devine le nombre

Atelier pratique



MISSION

Codez le jeu classique : l'ordinateur tire un nombre au hasard, l'utilisateur doit le trouver en un minimum d'essais.

Étapes à réaliser

1. Inclure `stdlib.h` et `time.h`, initialiser le générateur avec `srand(time(NULL))`.
2. Tirer un nombre aléatoire entre 1 et 100 avec `rand() % 100 + 1`.
3. Boucler tant que l'utilisateur n'a pas trouvé : lire son essai, comparer, dire "trop petit" ou "trop grand".
4. Compter le nombre de tentatives et l'afficher quand l'utilisateur trouve.
5. Encapsuler la logique de jeu dans une fonction `jouer(int secret)` qui retourne le nombre de tentatives.



ASTUCE ·

Bonus : proposer plusieurs niveaux de difficulté en variant la borne max (50, 100, 1000).

Bilan du Jour 3

Ce que vous savez maintenant faire



ACQUIS DE LA JOURNÉE

- Utiliser if/else et switch pour faire des choix
- Écrire des boucles for, while et do...while sans boucle infinie
- Définir et appeler des fonctions avec paramètres et retour
- Manipuler des tableaux et des chaînes de caractères
- Comprendre la différence passage par valeur / par adresse
- Lire et écrire les opérateurs *, & et utiliser des pointeurs simples



PROCHAINE ÉTAPE

Demain, on attaque la mémoire en profondeur. Vous découvrirez la différence entre la pile (stack) et le tas (heap), comment allouer dynamiquement avec malloc/free, et pourquoi on peut produire des bugs particulièrement vicieux : fuites mémoire, segfault, double free.

Ensuite, on commencera enfin C++ : comment passer du C au C++, les classes, et la POO.

Nous reverrons les pointeurs sous un angle différent — ils sont au cœur du Jour 4.